

EXAMEN CCP 2026

Gestion de Boutique

Laravel · MySQL · API REST · Swagger · React

Réalisé en binôme par

Aïssatou Niass & Birame Ndiaye

Formateur : Mr Tine

Sommaire du document

Ce document accompagne les deux dépôts GitHub du projet (backend Laravel et frontend React) réalisés dans le cadre de l'examen « Gestion de boutique » du CCP 2026. Il présente les vues de l'application frontend, la documentation Swagger de l'API, les restrictions d'accès par rôle, ainsi que les liens vers les dépôts du projet.

1. Présentation des vues du frontend (React)
2. Présentation de la documentation Swagger (Backend)
3. Vues correspondant aux différents rôles
4. Liens GitHub des dépôts
5. Conclusion

1. Présentation des vues du frontend (React)

Le frontend est une application React (Vite) entièrement séparée du backend, hébergée dans son propre dépôt GitHub. Elle consomme l'API REST Laravel via la bibliothèque axios, et la navigation entre les pages est gérée par react-router-dom, sans rechargement de page.

Accueil (/)

Page de bienvenue présentant l'application aux visiteurs.

Liste des produits (/produits)

Tableau listant l'ensemble des produits avec leur nom, prix, stock et catégorie associée. Chaque ligne propose un lien « Voir » vers le détail du produit.

Détail d'un produit (/produits/:id)

Affiche le nom, le prix, le stock, la catégorie et la description complète du produit sélectionné, avec un lien de retour vers la liste.

Liste des catégories (/categories)

Affiche toutes les catégories disponibles, avec leur description et le nombre de produits qui leur sont rattachés.

Une barre de navigation commune, présente sur toutes les pages, permet de circuler librement entre Accueil, Produits et Catégories. L'interface a été conçue pour rester lisible aussi bien sur ordinateur que sur mobile.

2. Présentation de la documentation Swagger (Backend)

La documentation de l'API est générée automatiquement avec le package **darkaonline/l5-swagger**, à partir d'annotations PHP (attributs #[OA\...]) placées directement dans les contrôleurs API. Elle est accessible à l'adresse suivante, une fois le serveur Laravel démarré :

<http://127.0.0.1:8000/api/documentation>

Organisation de la documentation

Les endpoints sont regroupés en trois catégories (tags), correspondant chacune à une entité du modèle de données :

Tag	Endpoints	Description
Catégories	GET/POST /categories GET/PUT/DELETE /categories/{id}	Gestion complète des catégories
Produits	GET/POST /produits GET/PUT/DELETE /produits/{id}	Gestion complète des produits
Acheteurs	GET/POST /acheteurs GET/PUT/DELETE /acheteurs/{id} POST /acheteurs/{id}/acheter	Gestion des acheteurs et enregistrement des achats

Contenu de chaque annotation

- Un résumé de l'action réalisée par l'endpoint
- Les paramètres attendus (par exemple l'identifiant dans l'URL)
- Le corps de requête attendu pour les méthodes POST/PUT, avec des exemples concrets
- Les codes de réponse possibles : 200 (succès), 201 (créé), 204 (supprimé), 404 (non trouvé), 422 (erreur de validation)

L'interface Swagger UI permet également de tester chaque endpoint en conditions réelles grâce au bouton « Essaie-le » (Try it out), sans avoir besoin d'un outil externe comme Postman : la requête, la réponse du serveur et les en-têtes HTTP s'affichent directement dans le navigateur.

3. Vues correspondant aux différents rôles

L'application distingue trois rôles utilisateurs : **employé**, **gestionnaire** et **admin**. Ces restrictions s'appliquent à la fois sur les routes (via le middleware personnalisé CheckRole) et sur l'affichage des vues Blade (les boutons d'action apparaissent ou disparaissent selon le rôle de la personne connectée).

Rôle	Droits
Employé	Consulter les catégories, produits et acheteurs (lecture seule). Enregistrer un nouvel achat depuis la fiche d
Gestionnaire	Tout ce que peut faire un employé, plus la création, la modification et la suppression des catégories, produit
Admin	Tout ce que peut faire un gestionnaire, plus la gestion des utilisateurs et de leurs rôles.

Toute tentative d'accès direct à une URL non autorisée pour le rôle connecté renvoie une erreur **403** — **Accès non autorisé**, aussi bien côté navigation Blade que côté API.

4. Liens GitHub des dépôts

Dépôt backend (Laravel)

<https://github.com/niass-dev2026/boutique-backend-NIASS-NDIAYE>

Dépôt frontend (React)

<https://github.com/niass-dev2026/boutique-frontend-NIASS-NDIAYE>

Chaque dépôt contient un fichier README.md détaillant la procédure d'installation, les comptes de test et les URL utiles. L'historique Git des deux dépôts reflète les contributions des deux membres du binôme, tous deux ajoutés comme collaborateurs.

5. Conclusion

Ce projet nous a permis de mettre en pratique, de bout en bout, les compétences visées par l'examen CCP 2026 : la conception d'un modèle de données relationnel, le développement d'un backend Laravel structuré autour de rôles utilisateurs, l'exposition d'une API REST documentée avec Swagger, et la consommation de cette API par une application front-end React totalement découplée.

Le travail en binôme, avec des dépôts GitHub partagés, nous a également permis de nous exercer à la collaboration en équipe sur un même projet : gestion des permissions, revue mutuelle du code et répartition des tâches entre backend, API et frontend.

L'ensemble des fonctionnalités demandées dans l'énoncé — authentification, gestion des catégories, produits et acheteurs, restrictions par rôle, API REST documentée et interface React — a été implémenté, testé manuellement et validé avant la remise finale.

Merci de l'attention portée à ce travail.

Aïssatou Niass & Birame Ndiaye — Examen CCP 2026